

## REDES DE COMPUTADORAS

### Trabajo Práctico N° 5

# Preguntas de repaso — Capítulo 20

WireGuardians

Viberti, Benjamin<sup>1</sup>

Espinoza Sutta, Aaron Alejandro<sup>1</sup>

Cleri, Juan Ignacio<sup>1</sup>

Pineda, Juan Ignacio<sup>1</sup>

Grafi3n, Atilio Leonel<sup>1</sup>

Badenes, Tom3s<sup>1</sup>

Oviedo, Ignacio Nicolas<sup>1</sup>

Mendez, Jorge Nicolas<sup>1</sup>

05/10/2026

<sup>1</sup>{b.viberti,aaron.espinoza\_4500,ignacio.cleri,juan.ignacio.pineda,atilio.grafion,tomasbadenes,  
ignacio.oviedo.239,jorge.mendez}@mi.unc.edu.ar

**Objetivo** — Resolver las preguntas de repaso del Cap3tulo 20 de [1], sobre protocolos de transporte: direccionamiento, multiplexaci3n, control de flujo por cr3ditos, establecimiento de conexi3n y los protocolos TCP y UDP.

---

## 20. Preguntas de repaso — Cap3tulo 20

### 20.1. ¿Qu3 elementos de direccionamiento son necesarios para especificar un usuario de servicio de transporte (TS) destino?

Para identificar al usuario destino se necesita la siguiente informaci3n:

- Identificaci3n del usuario.
- Identificaci3n de la entidad de transporte.
- Direcci3n de la estaci3n.
- N3mero de la red.

En la pr3ctica la direcci3n se expresa como el par (estaci3n, puerto), llamado *socket* en TCP, donde el puerto identifica a un usuario TS particular dentro de la estaci3n.

Normalmente hay una 3nica entidad de transporte por estaci3n, por lo que no hace falta identificarla. Si hubiera m3s de una (por ejemplo TCP y UDP), la direcci3n debe indicar adem3s el tipo de protocolo de transporte.

### 20.2. Describa cuatro estrategias por las que un usuario TS emisor pueda averiguar la direcci3n de un usuario TS receptor.

Hay dos estrategias est3ticas y dos din3micas:

Est3ticas:

- 1) **El usuario conoce previamente la direcci3n que desea utilizar:** es una configuraci3n del sistema. Sirve para procesos que interesan s3lo a unos pocos usuarios y que no deben ser conocidos por todos.
- 2) **Direcciones conocidas:** a algunos servicios de uso com3n se les asignan direcciones fijas conocidas por todos, por ejemplo un servidor FTP o SMTP.

Dinámicas:

- 1) **Servidor de nombres:** el usuario solicita mediante un nombre genérico o global la dirección de un usuario. El servidor de nombres devuelve la dirección, y luego la entidad de transporte establece la conexión. Es útil para servicios que cambian de localización de vez en cuando, por ejemplo para balancear la carga.
- 2) **Creación del proceso en el momento de la solicitud:** el emisor envía una petición a una dirección bien conocida, donde un proceso del sistema crea el proceso destino y devuelve su dirección. Por ejemplo, una cliente puede pedirle a un gestor de trabajos remoto que lance un programa de simulación en un servidor.

### 20.3. Explique el uso de la multiplexación en el contexto de un protocolo de transporte.

El protocolo de transporte utiliza multiplexación en dos sentidos respecto de los protocolos de capas superiores y respecto de los servicios de red que usa.

Respecto de las capas superiores, el protocolo de transporte multiplexa y demultiplexa múltiples usuarios sobre el mismo protocolo de transporte, distinguiéndose mediante números de puerto o puntos de acceso al servicio. En TCP, como una conexión queda determinada por los *sockets* origen y destino, un mismo puerto puede admitir múltiples conexiones, cada una con un puerto diferente. UDP incorpora a IP esta capacidad de direccionamiento de puerto.

Respecto de los servicios de red, la entidad de transporte puede usar multiplexación hacia arriba, que consiste en multiplexar múltiples conexiones sobre una única conexión de la capa inferior, o multiplexación hacia abajo, que consiste en dividir una única conexión entre múltiples conexiones de la capa inferior. Por ejemplo, en el caso de redes X.25, si un solo circuito tiene el rendimiento suficiente para varios usuarios conviene multiplexarlos sobre él. Por otra parte, cada circuito X.25 está restringido en números de secuencia, y en redes de alta velocidad y gran retardo podría requerirse un rango mayor, por lo que dividir la conexión entre varios circuitos puede mejorar el rendimiento.

### 20.4. Describa brevemente el esquema de créditos utilizado por TCP para el control de flujo.

Es un control de flujo extremo a extremo donde el receptor le indica explícitamente al emisor cuántos datos puede enviar. Cada octeto tiene un número de secuencia y los segmentos llevan tres campos: número de secuencia (SN, el del primer octeto de datos del segmento), número de confirmación (AN) y ventana (W). Un segmento con ( $AN = i$ ,  $W = j$ ) significa:

- Se confirman todos los octetos hasta  $i - 1$ ; el siguiente esperado es el  $i$ .
- Se concede crédito para enviar  $j$  octetos más, del  $i$  al  $i + j - 1$ .

El emisor avanza el borde final de su ventana a medida que transmite y el borde inicial solo cuando recibe crédito nuevo (la ventana se achica al transmitir y se amplía con cada crédito). El receptor no está obligado a confirmar cada segmento: puede enviar una confirmación acumulada.

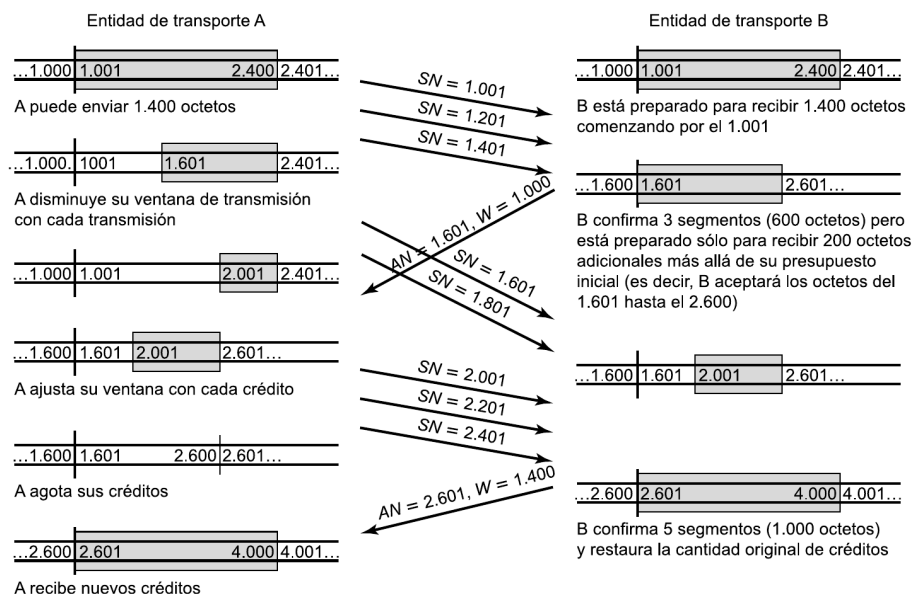


Figura 1. Ejemplo del mecanismo de asignación de crédito de TCP.

El receptor puede ser *conservador* (conceder solo el espacio libre real en su memoria temporal) u *optimista* (conceder espacio que espera liberar, mejorando el rendimiento con retardos grandes; pero si el emisor es más rápido que el receptor se descartan segmentos y hay que retransmitir, lo que complica el protocolo).

## 20.5. ¿Cuál es la diferencia principal entre el esquema de créditos de TCP y el esquema de control de flujo de ventana deslizante utilizada por muchos otros protocolos, como por ejemplo HDLC?

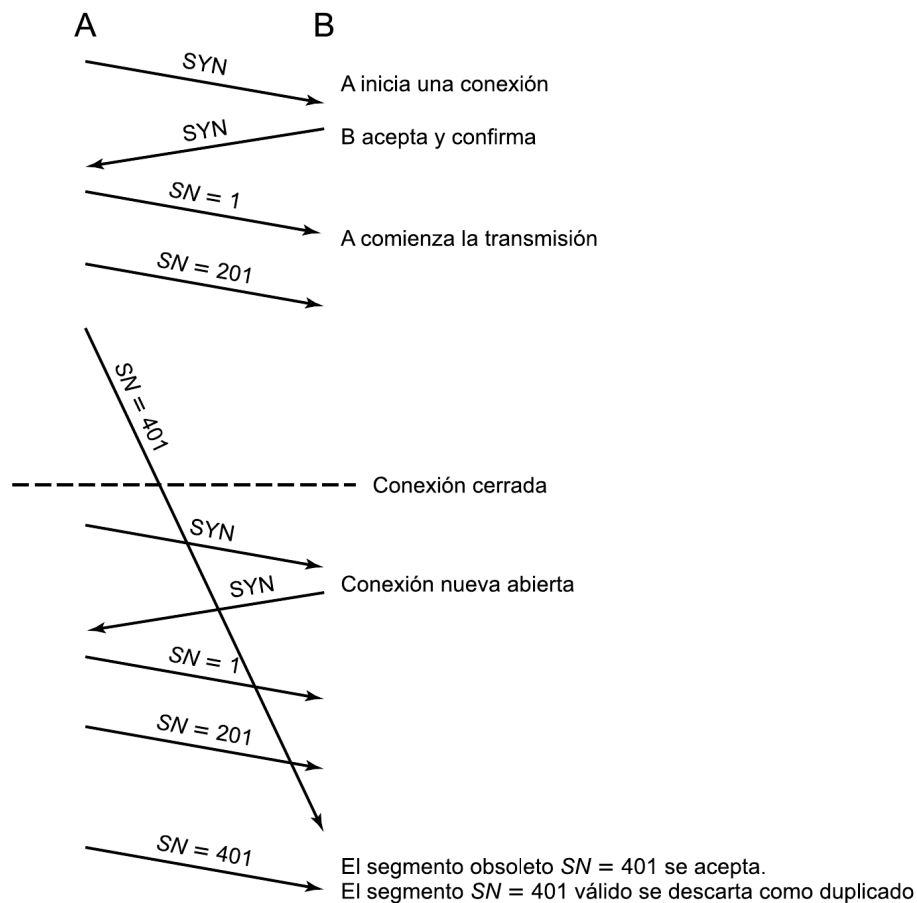
En la ventana deslizante fija (como en X.25) confirmar y dar permiso de envío son lo mismo: cada confirmación avanza automáticamente una ventana de tamaño fijo, y para frenar al emisor el receptor retiene las confirmaciones. En el esquema de créditos de TCP la **confirmación y el control de flujo están desacoplados**: se puede confirmar un segmento sin conceder crédito nuevo ( $W = 0$ ) y conceder crédito sin confirmar datos nuevos. Además, la ventana es variable y se mide en octetos, no en tramas.

Esto pesa en redes no fiables: con ventana fija, si el receptor frena reteniendo confirmaciones, el emisor no puede distinguir si la falta de confirmaciones se debe al control de flujo o a una pérdida. Con créditos el receptor confirma aunque no conceda crédito, y la pérdida de un segmento de confirmación/crédito se corrige con las confirmaciones posteriores. El único riesgo es un bloqueo mutuo si se pierde el ( $AN = i, W = j$ ) que reabre una ventana cerrada con  $W = 0$ ; se evita con un temporizador de ventana que obliga a reenviar un segmento al expirar.

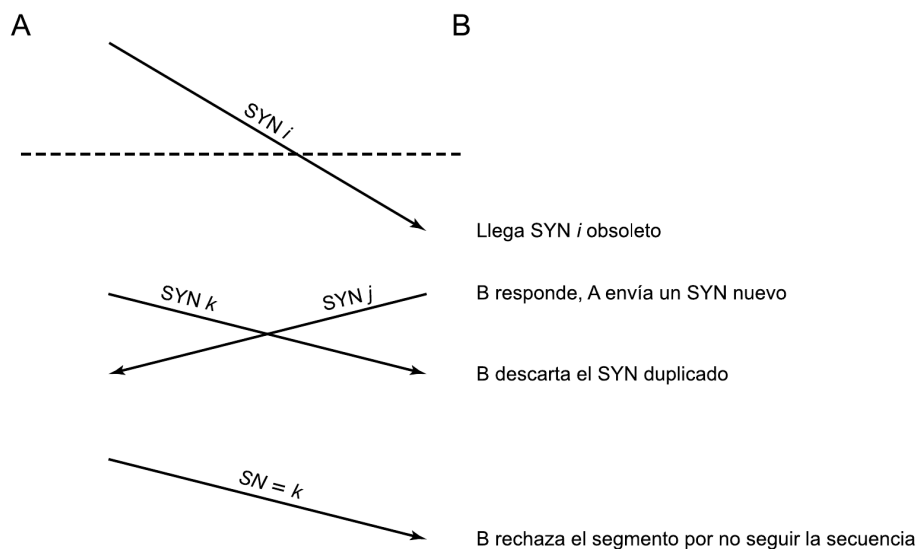
## 20.6. Explique los mecanismos de diálogo en dos y tres pasos.

**Diálogo en dos pasos:** A envía un SYN (solicitud de conexión); si B está en LISTEN, responde con un SYN que funciona como confirmación y pasa a ESTAB, y A pasa a ESTAB al recibirlo. La pérdida de un SYN se cubre con un temporizador de retransmisión, y los SYN duplicados se ignoran una vez establecida la conexión. Basta con un servicio de red fiable, pero sobre una red no fiable falla:

- Un segmento de datos obsoleto de una conexión anterior puede llegar durante una nueva y aceptarse en lugar del válido, que se descarta como duplicado. Para evitarlo, cada conexión empieza con un número de secuencia distinto, que se anuncia en el SYN (SYN  $i$ ).
- Eso no resuelve un SYN  $i$  obsoleto: B lo toma como una solicitud nueva y responde, descarta el SYN real de A como duplicado, y ambos creen tener una conexión válida con números de secuencia distintos, por lo que B rechaza los datos de A.



**Figura 2.** Diálogo en dos pasos: un segmento de datos obsoleto se acepta en una conexión nueva.



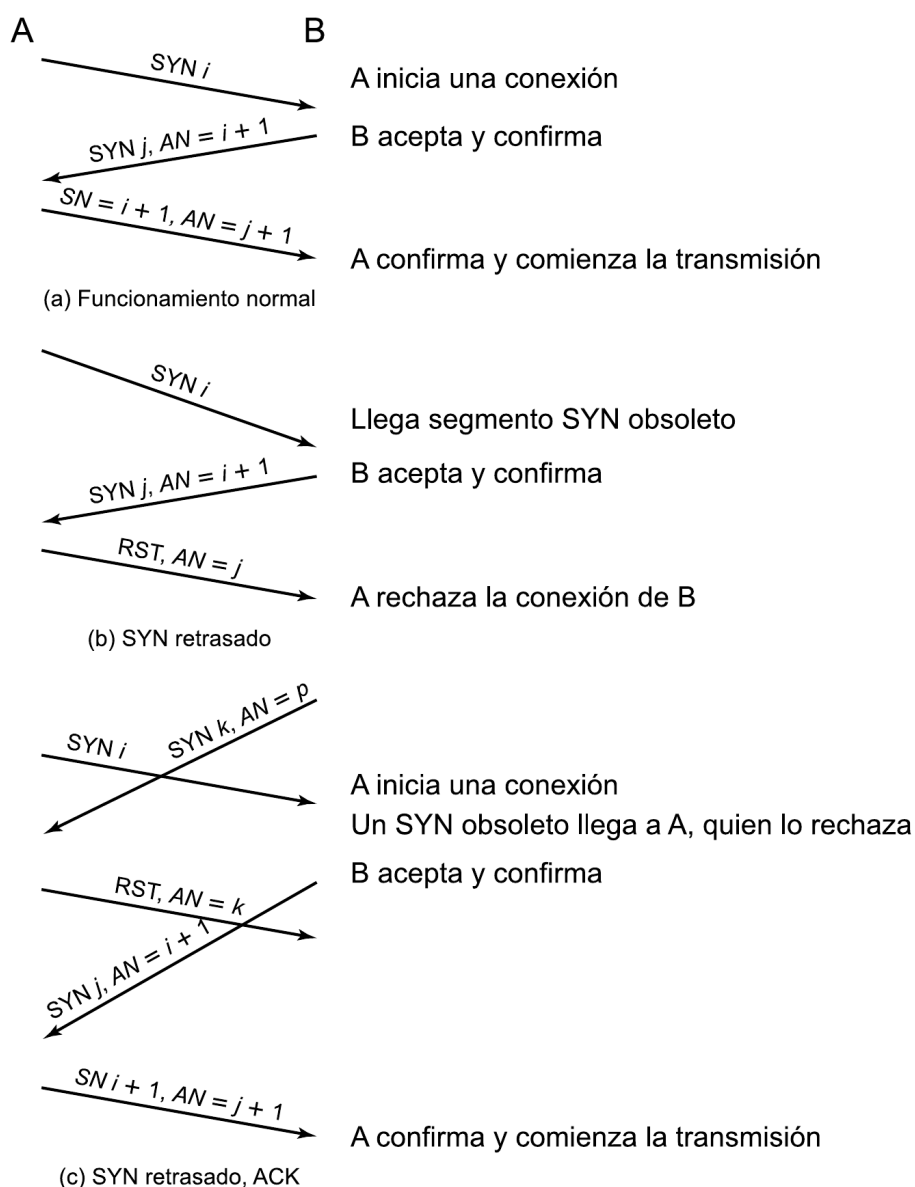
**Figura 3.** Diálogo en dos pasos: un SYN obsoleto desincroniza los números de secuencia.

**Diálogo en tres pasos (usado por TCP):** cada extremo confirma explícitamente el SYN y el número de secuencia inicial (ISN) del otro. El SYN ocupa el número  $i$ , por lo que el primer octeto de datos es el  $i + 1$ :

- 1) A → B: SYN, SN =  $i$
- 2) B → A: SYN, SN =  $j$ , AN =  $i + 1$
- 3) A → B: SN =  $i + 1$ , AN =  $j + 1$  (es el primer segmento de datos de A)

Se agrega el estado SYN RECEIVED, para no declarar la conexión establecida hasta que ambos SYN estén confirmados, y el segmento RST. La regla es enviar RST si la conexión todavía no está en ESTAB y llega un ACK inválido (que no referencia nada enviado):

- Si un SYN  $i$  obsoleto llega a B, este responde SYN  $j$ ,  $AN = i + 1$ ; A no pidió esa conexión y responde RST,  $AN = j$ . El AN en el RST evita que un RST obsoleto cancele una apertura legítima.
- Si un SYN/ACK obsoleto (SYN  $k$ ,  $AN = p$ ) llega a A mientras abre una conexión, A responde RST,  $AN = k$  y la apertura real sigue sin problemas, porque las confirmaciones llevan números de secuencia.



**Figura 4.** Diálogo en tres pasos: (a) funcionamiento normal, (b) SYN retrasado, (c) SYN/ACK obsoleto durante una apertura.

## 20.7. ¿Cuál es el beneficio del mecanismo de diálogo en tres pasos?

Su beneficio es que evita que los segmentos SYN duplicados u obsoletos, que llegan con retraso desde conexiones anteriores, generen conexiones falsas o confundan una conexión nueva. Para lograrlo, cada extremo confirma explícitamente el SYN y el número de secuencia inicial del otro antes de dar la conexión por establecida.

**20.8. Defina las características de urgencia y forzado de TCP.**

**20.9. ¿Qué es una opción en los criterios de implementación de TCP?**

**20.10. ¿Cómo puede utilizarse TCP para tratar la congestión de red o de interconexión de red?**

**20.11. ¿Qué proporciona UDP que no ofrezca IP?**

UDP añade a IP dos cosas esenciales:

Puertos: IP lleva el paquete hasta la computadora destino. En cambio UDP usa números de puerto para entregarlo a la aplicación exacta de destino.

Detección de errores: IP solo revisa que su propia cabecera esté bien. UDP calcula un Checksum que verifica que el contenido de los datos no se haya corrompido en el camino.

## **Bibliografía**

[1] W. Stallings, *Comunicaciones y redes de computadores*, 7.<sup>a</sup> ed. Madrid: Pearson Educación, 2004.